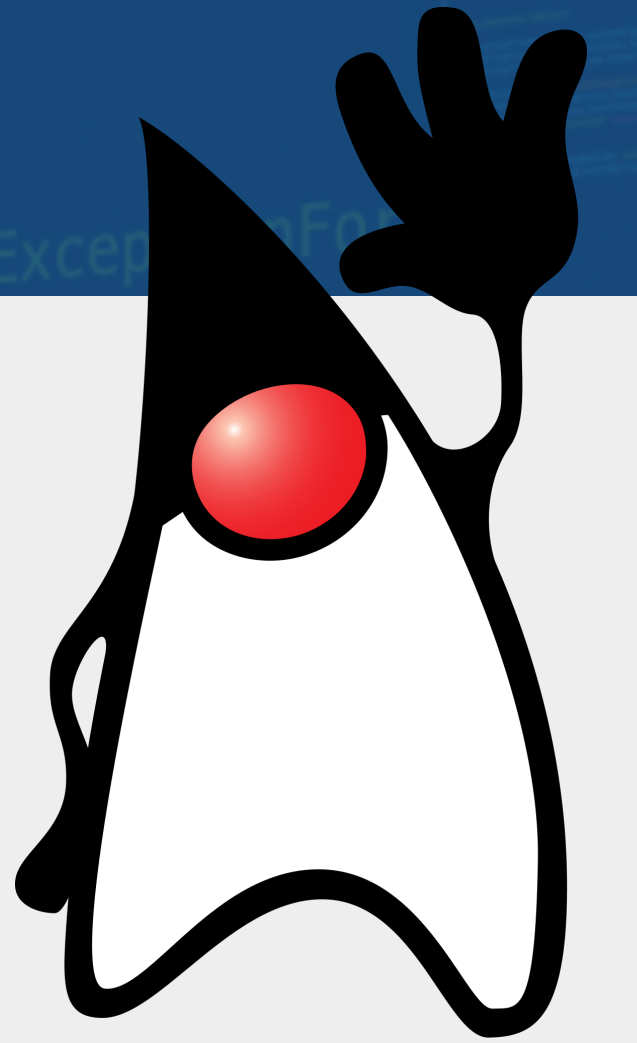


Java Programlama Dili

16. Bölüm Kurucular

Akın Kaldıroğlu



Amaç



- Bu bölümde nesne oluştururken çağrılan özel metotlar olan kurucular (constructors) ele alınacaktır



- **Kurucular**

- Varsayılan Kurucu
- Akıllı Kurucu
- Kurucuların Çoklu Kullanımı
- Kurucuda Metot Çağrısı

- **Bazı Noktalar**

- null Geçilen Kurucu

- Kurucular ve Yeni Eklenen Alanlar

- Fake Constructor

- Oluşturulması Bitmeden Nesneden Hizmet Almak

- Referanssız Nesneler

- **Ödevler**

- **Bundan Sonra?**

Kurucular



Nesnenin Durumu



- Şu ana kadar nesneleri oluşturduktan sonra bazen doğrudan erişim ile bazen de set metotları ile nesnelerin durumlarını istenilen hale getirdik
- Fakat bu zahmetli bir iştir, alan sayısı kadar satır yazmak gerekir
- Ve halen nesnelerin anlamsız durumda oldukları an söz konusudur
- Bunun çözümü **kurucu (constructor)** kullanmaktır

```
Person person = new Person();  
// Not a meaningful state!  
person.tckn = "1";  
person.firstName = "Zeynep";  
person.lastName = "Kaya";
```

```
Person person = new Person();  
// Not a meaningful state!  
person.setTckn("1");  
person.setFirstName("Zeynep");  
person.setLastName("Kaya");
```

Tree



- `noConst.Tree`

Kurucu (Constructor) - I



- Nesne oluşturulurken çağrılan özel metoda **kurucu** (ya da **yapılandırıcı (constructor)**) denir
- Kurucu metotlar sadece nesne oluştururken ve **new** ile çağrılır
- Java'da kurucu çağrısı yapmadan nesnesi oluşturulabilen çok az tip vardır:
 - **String**, dizi (array), ilkel tiplerin nesne halleri (wrappers), vs.
- Diğer her türlü nesne, ancak ve ancak kurucu çağrısı ile oluşturulur

Kurucu (Constructor) - II



- Kurucu çağrısı ile nesnenin alanlarına ilk değer atanabilir
- Bu amaçla kurucular parametre alacak şekilde tanımlanabilir ve çağrılırken argüman geçilir
- Böylece kurucu çağrısı yapıldığında nesnenin durumu anlamlı hale gelmiş olur
- Bu şekilde yaratılan nesnenin adresi referansa atandığında, durumu anlamlı ve sağlıklı halde olan nesne, kullanılabilir

Kurucu (Constructor) - III



- Kurucular özel metotlardır:
- Kurucu, içinde tanımlandığı sınıfın ismini alır
 - Dolayısıyla ismi, diğer metotların aksine, **Upper Camel Case**'e uyar
- Kurucuların dönüş tipi ve değeri yoktur, öncesine `void` bile yazılmaz
- Kurucular sadece nesne oluştururken ve **new** ile çağrılır

Kurucu (Constructor) - IV



- Öte yandan kurucular pek çok açıdan normal metot gibi davranır:
 - Kurucularda metot çağrısı vb. herhangi bir kod olabilir
 - Kurucular çoklu kullanılabilir (overloading)
 - Bu durum farklı nesne yaratma şekillerine karşılık gelir
 - Overload edilmiş kurucular birbirlerini çağırabilir
 - Bu konuyu, Java 25 ile gelen yeniliklerle birlikte, **this** konusunda ele alacağız

Kurucu (Constructor) - V



- Kurucularda, hiçbir şey döndürmedikleri için, genel olarak **return** cümlesi olmaz
- Varsa sadece **return;** olabilir, **return**'den sonra bir değer olamaz, aksi takdirde derleme hatası olur
- Kurucunun arayüzünde **void** de yazılmaz

Kurucu (Constructor) - VI



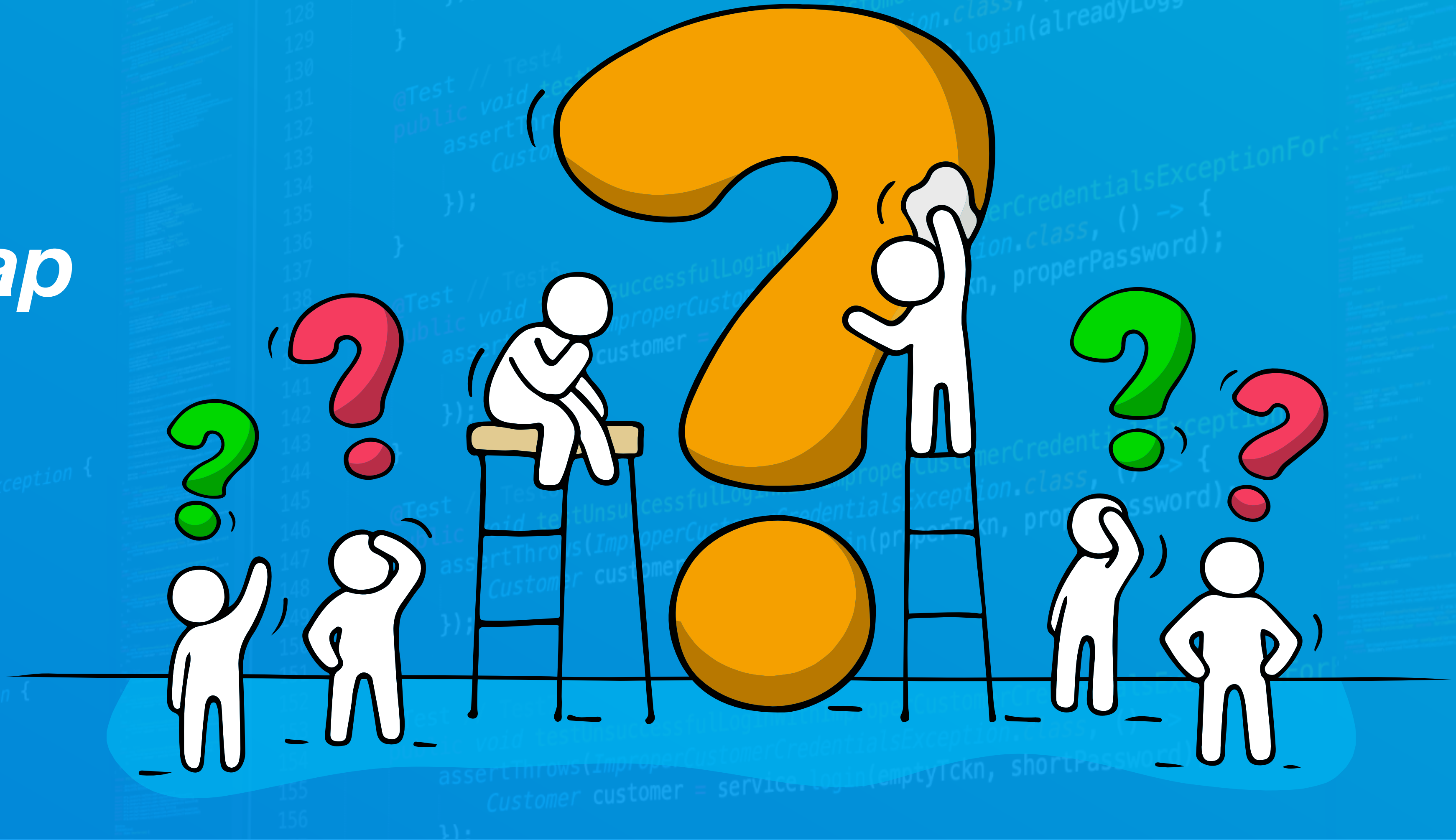
- Kurucular, üye metot (member method) değildir bu yüzden ileride göreceğimiz gibi, üye metotlar için geçerli olan miras kuralları, kurucular için geçerli değildir

Kurucu Tipleri



- Argüman alıp almamalarına göre iki tür kurucu vardır:
 - Hiçbir argüman almayan, **varsayılan/argümansız/aptal kurucu** (**default, no-argument (no-arg)** ya da **dummy constructor**)
 - Argüman alan kurucu, **argümanlı/akıllı kurucu** (**argument-constructor, arg-constructor** ya da **smart constructor**)

Soru ve Cevap Zamanı!





Varsayılan Kurucu

Varsayılan Kurucu - I



- Java'da kurucusuz sınıf olamaz!
- Java kaynak kodunda geliştirici tarafından sağlanmış bir kurucu olmasa da `class` dosyasında muhakkak en az bir tane kurucu olur
- Çünkü derleyici, `class` dosyasına bir tane varsayılan kurucu ekler
- Varsayılan kurucu, argüman almayan kurucudur ve derleyici tarafından sağlandığından içi boştur
- Şimdilik böyle kabul edelim!

Varsayılan Kurucu - II



- Derleyicinin sağladığı varsayılan kurucu hiçbir parametre almaz ve değer de atamaz

```
class Tree { // No constructor defined
    String type;
    float height;

    // Compiler defines a default constructor
    // Tree(){}
    void printInfo(){...}

    void main() {
        Tree tree = new Tree();
        tree.printInfo(); //height is 0.0 & type is null
        tree.type = "Oak";
        tree.height = 8.74f;
        tree.printInfo(); //height is 8.74 & type is Oak
    }
}
```



- `noConst.Tree`
- Bu örnekteki `Tree` sınıfında kurucu yoktur ama buna rağmen `new Tree()` çağrısı yapılabilmektedir

Varsayılan Kurucu - II



- Geliştirici isterse argüman almayan bir kurucu sağlar ve içinde nesnenin alanlarına ilk değer atamaları yapar
- Bu durumda derleyici tabii olarak varsayılan kurucu sağlamaz
 - Çünkü aynı kurucudan iki tane olamaz!
- Geliştirici gerekli başka kodları da argümansız kurucu içine koyabilir

```
class Tree {  
    String type;  
    float height;  
  
    Tree(){  
        type = "Pine";  
        height = 1.0f;  
        ...  
    }  
  
    void main(){  
        Tree tree1 = new Tree();  
        Tree tree2 = new Tree();  
        ...  
    }  
}
```

Tree



- `noArg.Tree`

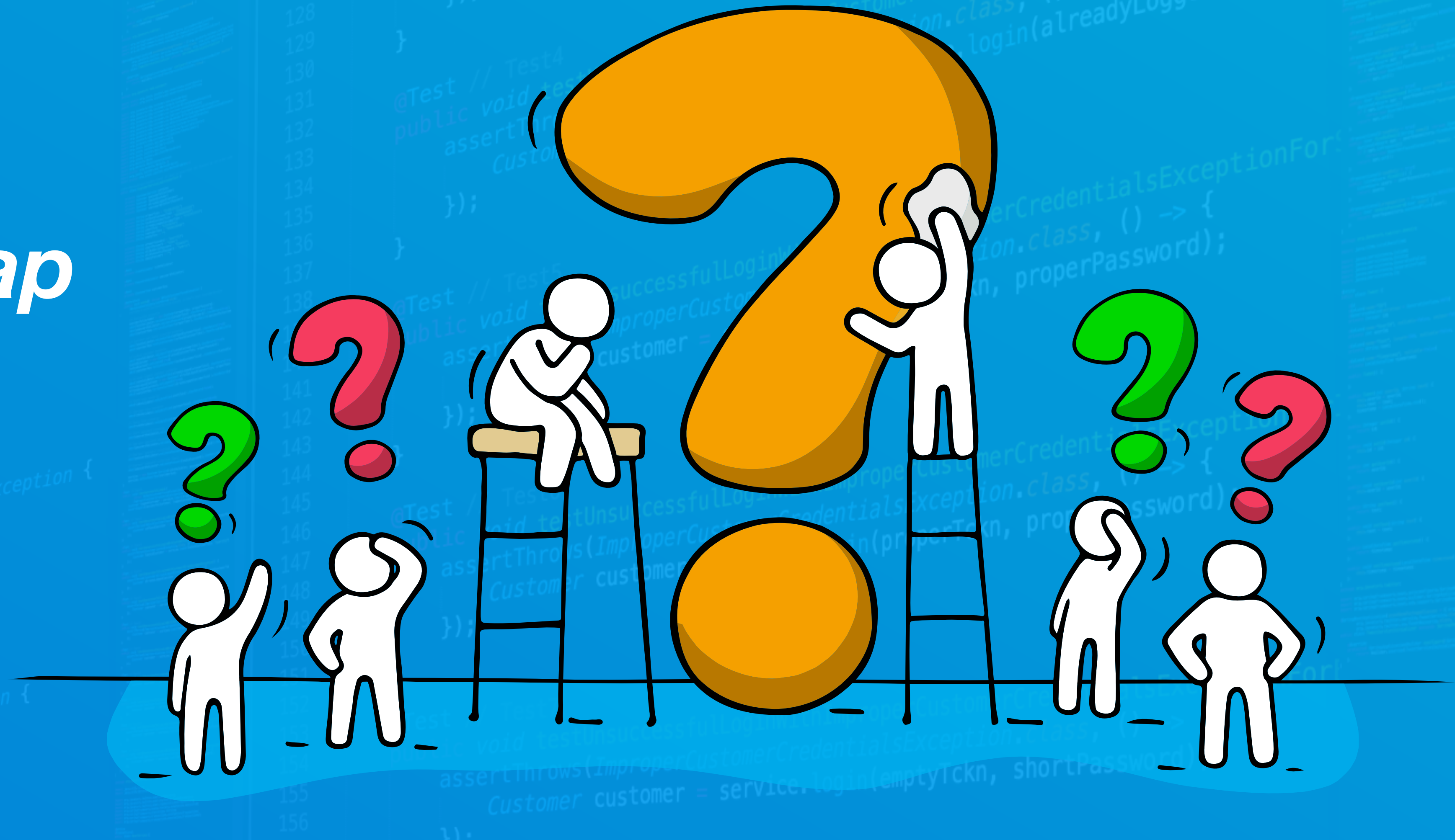
Varsayılan Kurucu - IV



- Bu yaklaşımın problemi, her **Tree** nesnesinin aynı değerlerle oluşturulmasıdır
- Yani argümansız kurucu, programcıya bir fayda sağlayamayabilir çünkü nesne oluşturulduktan sonra hâlâ nesnenin istenen duruma getirilmesi gereklidir

```
class Tree {  
    String type;  
    float height;  
  
    Tree(){  
        type = "Pine";  
        height = 1.0f;  
        ...  
    }  
  
    void main(){  
        Tree tree1 = new Tree();  
        tree1.type = "Oak";  
        tree1.height = 8.74f;  
  
        Tree tree2 = new Tree();  
        ...  
    }  
}
```

Soru ve Cevap Zamanı!





Akıllı Kurucu

Akıllı Kurucu - I



- Argümentsiz kurucunun çözemediğini akıllı kurucu çözer
- **Akıllı kurucu (smart constructor)**, dışarıdan argüman alır (**argument constructor**)
- Geçilen argümanlar nesnenin alanlarına atanarak, nesnenin anlamlı bir duruma sahip olması sağlanır
- Artık her nesne farklı değerlere sahip alanlarla yani durumla oluşturulabilir

Akıllı Kurucu - II



- Akıllı kurucu, alanlara ilk değer olarak atamak üzere dışarıdan argüman alır
- Artık her **Tree** nesnesi farklı değerlerle, farklı bir duruma sahip olacak şekilde oluşturulabilir

```
class Tree {  
    String type;  
    float height;  
  
    // A smart constructor  
    Tree(String newType, float newHeight){  
        type = newType;  
        height = newHeight;  
    }  
  
    void main(){  
        Tree tree1 = new Tree("Pine", 2.0f);  
        Tree tree2 = new Tree("Oak", 5.5f);  
    }  
}
```

Tree



- `argConst.Tree`

Akıllı ve Varsayılan Kurucu - I



- **Akıllı kurucu (smart constructor)**
sağlanıyorsa derleyici varsayılan kurucu sağlamaktan vazgeçer
- Bu durumda varsayılan kurucu çağrısı bir derleme hatasıdır
- **Soru:** Argüman alan bir kurucunun varlığında, derleyicinin varsayılan kurucuyu sağlamamasının sebebi ne olabilir?

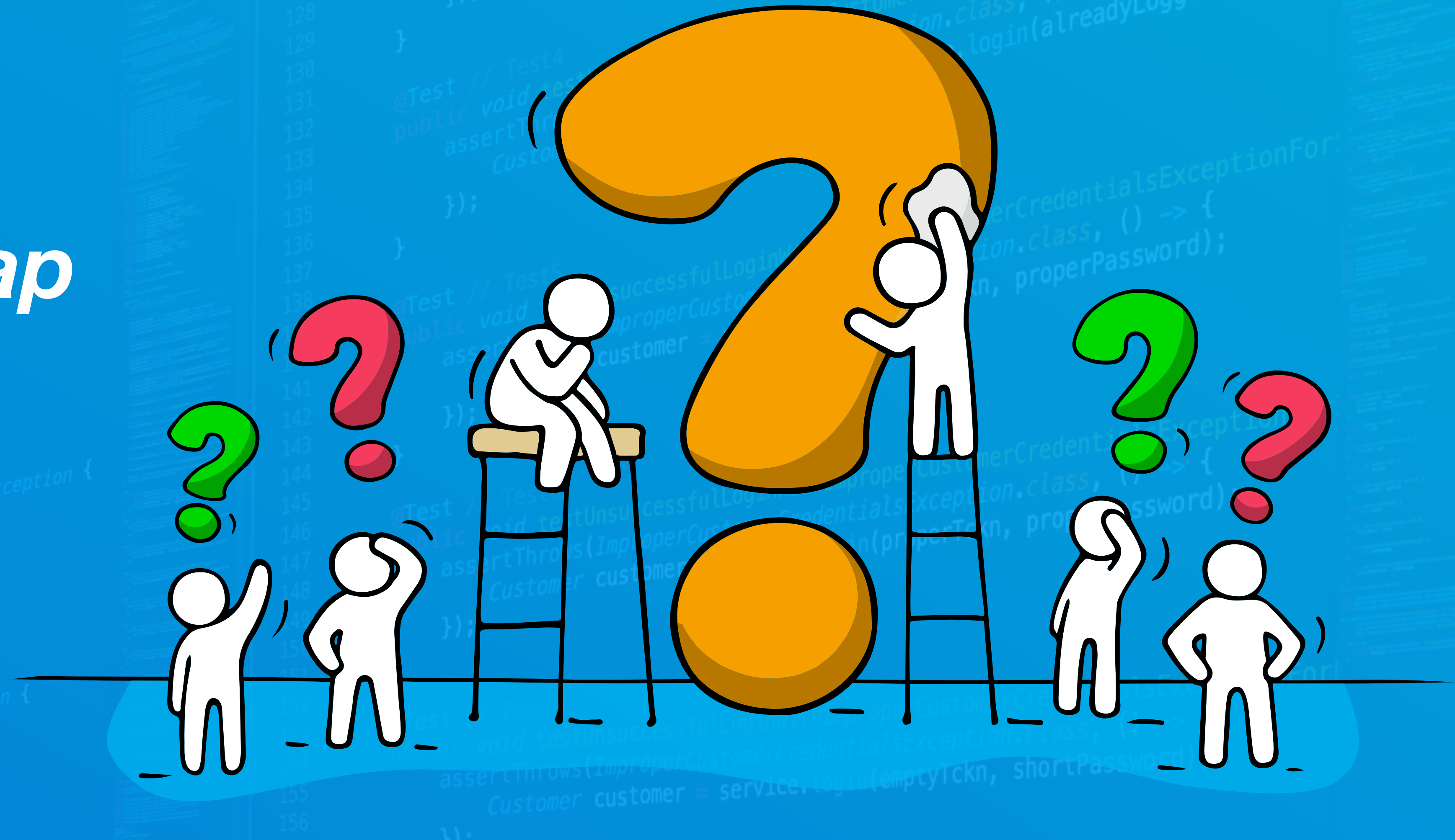
```
class Tree {  
    String type;  
    float height;  
  
    Tree(String newType, float newHeight){  
        type = newType;  
        height = newHeight;  
    }  
  
    void main(){  
        Tree tree1 = new Tree("Pine", 2.0f);  
        Tree tree2 = new Tree("Oak", 5.5f);  
  
        // Compiler error!  
        // No default constructor!  
        Tree tree3 = new Tree();  
    }  
}
```

Akıllı ve Varsayılan Kurucu - II



- Çünkü, geliştirici akıllı kurucu(lar) sağlıyorsa, varsayılan kurucunun olup olmaması da onun inisiyatifinde olmalıdır
- Yani geliştirici, nesnenin farklı yaratılma şekillerine karar verirken varsayılan kurucuyu istemeyebilir
- Bu yüzden derleyici, akıllı bir kurucunun olması durumunda varsayılan kurucuyu sağlamaktan vazgeçer

Soru ve Cevap Zamanı!





Kurucuların Çoklu Kullanımı

Kurucuların Çoklu Kullanımı - I



- Bir sınıfta birden fazla kurucu olabilir
- Bu durumda kurucular çoklu kullanılmıştır (**overloaded constructors**)
 - Çünkü kurucuların isimleri aynıdır ama aldığı parametreler farklı olmalıdır!
- Bir sınıfta birden fazla kurucunun olması, farklı nesne oluşturma şekillerinin dolayısıyla da farklı ilk durumların olduğunu gösterir

Kurucuların Çoklu Kullanımı - II



- Tasarım sırasında nesnelerin hangi kurucularla oluşturulacağına karar vermek gereklidir
- Aslen karar konusu, “bir sınıfın nesnesi hangi verilerle üretilebilmelidir?” sorusudur

```
class Employee {  
    int id;  
    String firstName;  
    String lastName;  
    LocalDate birthDate;  
    float salary;  
    String department;  
    String phone;  
    Address address;  
}
```

Kurucuların Çoklu Kullanımı - II



- **Employee**'de varsayılan kurucu yoktur ve akıllı iki kurucu farklı argümanlar almaktadır
- İlk kurucu, ikinci kurucudan bir fazla argüman ile nesneyi oluşturmaktadır

```
class Employee {
    int id;
    String firstName;
    String lastName;
    LocalDate birthDate;
    float salary;
    String department;
    String phone;
    Address address;
    // No default constructor defined

    Employee(int newId, String newFirstName, String newLastName,
              LocalDate newBirthDate, float newSalary, String newPhone,
              Address newAddress){
        id = newId;
        firstName = newFirstName;
        lastName = newLastName;
        birthDate = newBirthDate;
        salary = newSalary;
        phone = newPhone;
        address = newAddress;
    }

    Employee(int newId, String newFirstName, String newLastName,
              LocalDate newBirthDate, float newSalary,
              String newDepartment) {
        id = newId;
        ...
        department = newDepartment;
    }
}
```

Employee

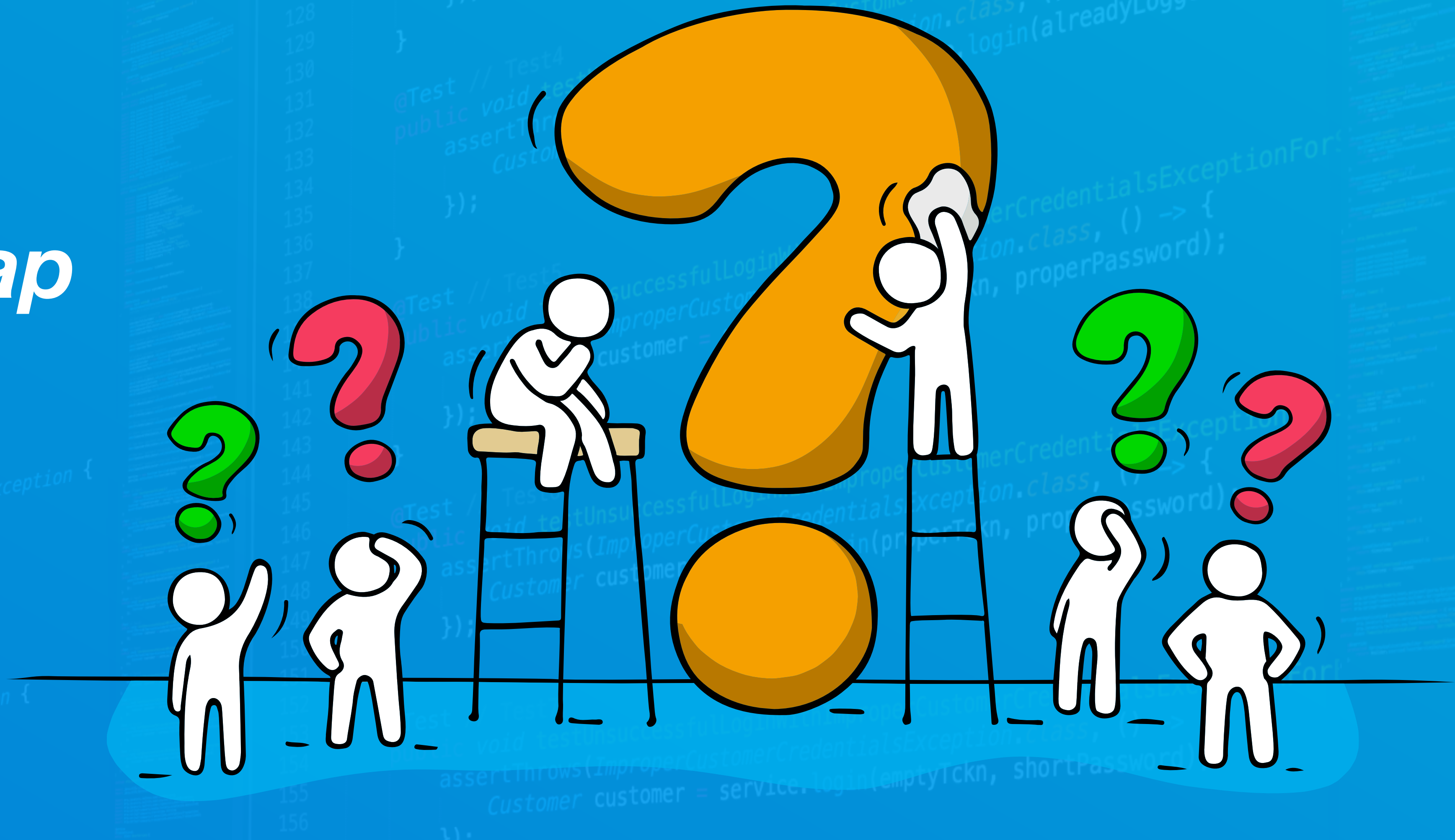


- `overloaded.Employee`



- `overloaded.Tree`
- **Tree** örneğinde 4 ayrı kurucu dolayısıyla dört farklı **Tree** oluşturma şeklinin olduğunu gözlemleyin

Soru ve Cevap Zamanı!





Kurucuda Metot Çağrısı

Kurucuda Metot Çağrısı



- Kurucularda metot çağrısı yapılabilir
- Kurucuda çağrılan metotlar genelde nesnenin durumu için gerekli değerleri üretir ya da farklı kaynaklardan toplar vs.
- Bu tür metotlar, argüman alan ya da almayan kurucularda kullanılabilir

```
class Tree {  
    String type;  
    float height;  
  
    Tree(){  
        type = produceType();  
        height = produceHeight();  
    }  
  
    String produceType(){  
        ...  
    }  
  
    float produceHeight(){  
        ...  
    }  
}
```

```
class Tree {  
    String type;  
    float height;  
  
    Tree(String newType){  
        type = newType;  
        height = produceHeight();  
    }  
  
    float produceHeight(){  
        ...  
    }  
}
```

RandomTree



- `noArg.RandomTree`

Bazı Noktalar





null Geçilen Kurucu

null Geçilen Kurucu - I



- Kuruculara `null` geçerek nesne oluşturmaktan kaçınmak gereklidir
- Null pointer exception!
- Bunun yerine `null` değer geçilecek parametreye sahip olmayan kurucu oluşturmalı ve onu kullanmalıdır

```
class Employee {  
    ...  
    String department;  
  
    Employee(int newId, String newFirstName,  
             String newLastName, LocalDate newBirthDate,  
             float newSalary, String newDepartment) {  
        ...  
    }  
  
    Employee(int newId, String newFirstName,  
             String newLastName, LocalDate newBirthDate,  
             float newSalary) {  
        ...  
    }  
}
```

```
Employee e = new Employee(1, "Ali", "Ozgun", dob, 12000f, null);  
// The other constructor should be called instead of passing `null`  
Employee e = new Employee(1, "Ali", "Ozgun", dob, 12000f);
```

null Geçilen Kurucu - II



- `null` geçilecek parametreye sahip olmayan kurucunun çağırılması halinde, ilk değer atanmayan alana anlamlı bir ilk değer verilmelidir

```
class Employee {  
    ...  
    String department = "No department yet!";  
  
    Employee(int newId, String newFirstName,  
             String newLastName, LocalDate newBirthDate,  
             float newSalary, String newDepartment) {  
        ...  
    }  
  
    Employee(int newId, String newFirstName,  
             String newLastName, LocalDate newBirthDate,  
             float newSalary) {  
        ...  
    }  
}
```

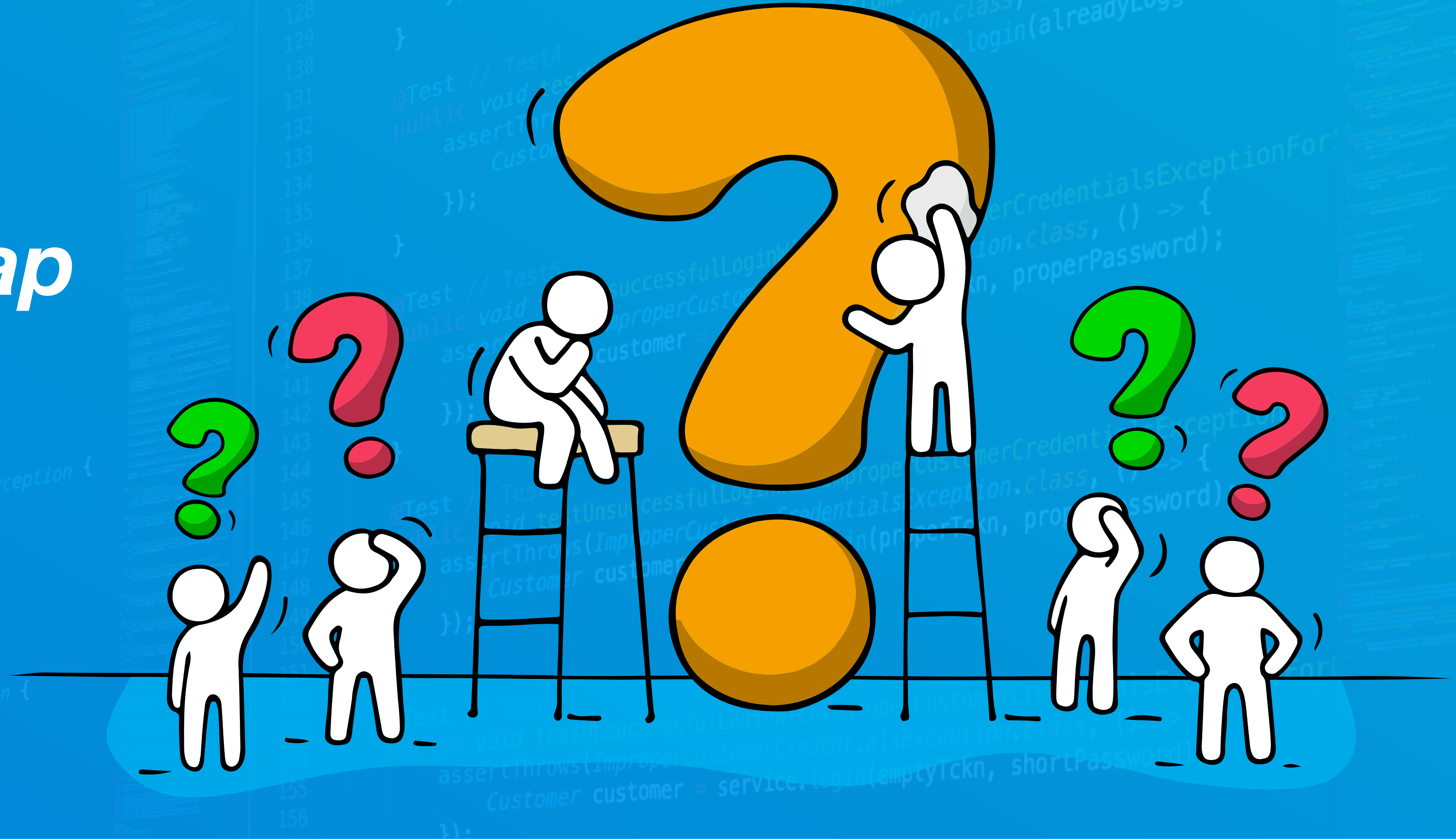
```
class Employee {  
    ...  
    String department;  
  
    Employee(int newId, String newFirstName,  
             String newLastName, LocalDate newBirthDate,  
             float newSalary, String newDepartment) {  
        ...  
    }  
  
    Employee(int newId, String newFirstName,  
             String newLastName, LocalDate newBirthDate,  
             float newSalary) {  
        ...  
        department = "No department yet!";  
    }  
}
```

Employee



- `problem.Employee`

Soru ve Cevap Zamanı!





Kurucular ve Yeni Eklenen Alanlar

Yeni Eklenen Alanlar - I



- Zaman zaman kurucuların varlığına rağmen nesnenin alanlarının tamamına kurucuda değer atanmadığı, nesnenin yaratıldığı yerde değer atandığı görülür

```
class Car {  
    String make;  
    String model;  
    String year;  
    int speed;  
    int distance;  
  
    Car(String newMake, String newModel, String newYear,  
        int newSpeed, int newDistance) {  
        make = newMake;  
        ...  
        distance = newDistance;  
    }  
}
```

```
Car car = new Car("Mercedes", "C200", "2024", 0, 0);
```

Yeni Eklenen Alanlar - II



- Bu, genelde nesneye sonradan alanların eklenmesi durumunda olur
- Yeni eklenen alanlara ilk değer ataması, kurucular yerine eklenen **set** metotlarıyla yapılır
- Bu ise hem düzeni bozar hem de hataya açık bir duruma sebep olur, çünkü **set** metotlarını çağırmak unutulabilir

```
class Car {
    String make;
    String model;
    String year;
    int speed;
    int distance;
    String color; // Added later
    boolean convertible; // Added later

    Car(String newMake, String newModel, String newYear,
        int newSpeed, int newDistance) {
        make = newMake;

        ...
        distance = newDistance;
        // No color and convertible initialization added!
    }

    // Setters added!
    void setColor(String color){...}
    void setConvertible(boolean b){...}
}
```

```
Car car = new Car("Mercedes", "C200", "2024", 0, 0);
car.setColor("Blue"); // Problem!
car.setConvertible(true); // Problem!
```

Yeni Eklenen Alanlar - III



- Aslında olması gereken yeni alanlara kurucularda ilk değer atanması için kurucuların parametre listesinin ve gerçekleştirilmesinin güncellenmesidir

```
class Car {  
    ...  
    String color; // Added later  
    boolean convertible; // Added later  
  
    Car(String newMake, String newModel, String newYear,  
        int newSpeed, int newDistance,  
        String newColor, boolean newConvertible) {  
        make = newMake;  
        ...  
        distance = newDistance;  
        color = newColor;  
        convertible = newConvertible;  
    }  
}
```

```
Car car = new Car("Mercedes", "C200", "2024", 0, 0, "Blue", true);
```

Yeni Eklenen Alanlar - IV



- Gerekirse yeni alanlara değer atayan yeni bir kurucu da eklenebilir

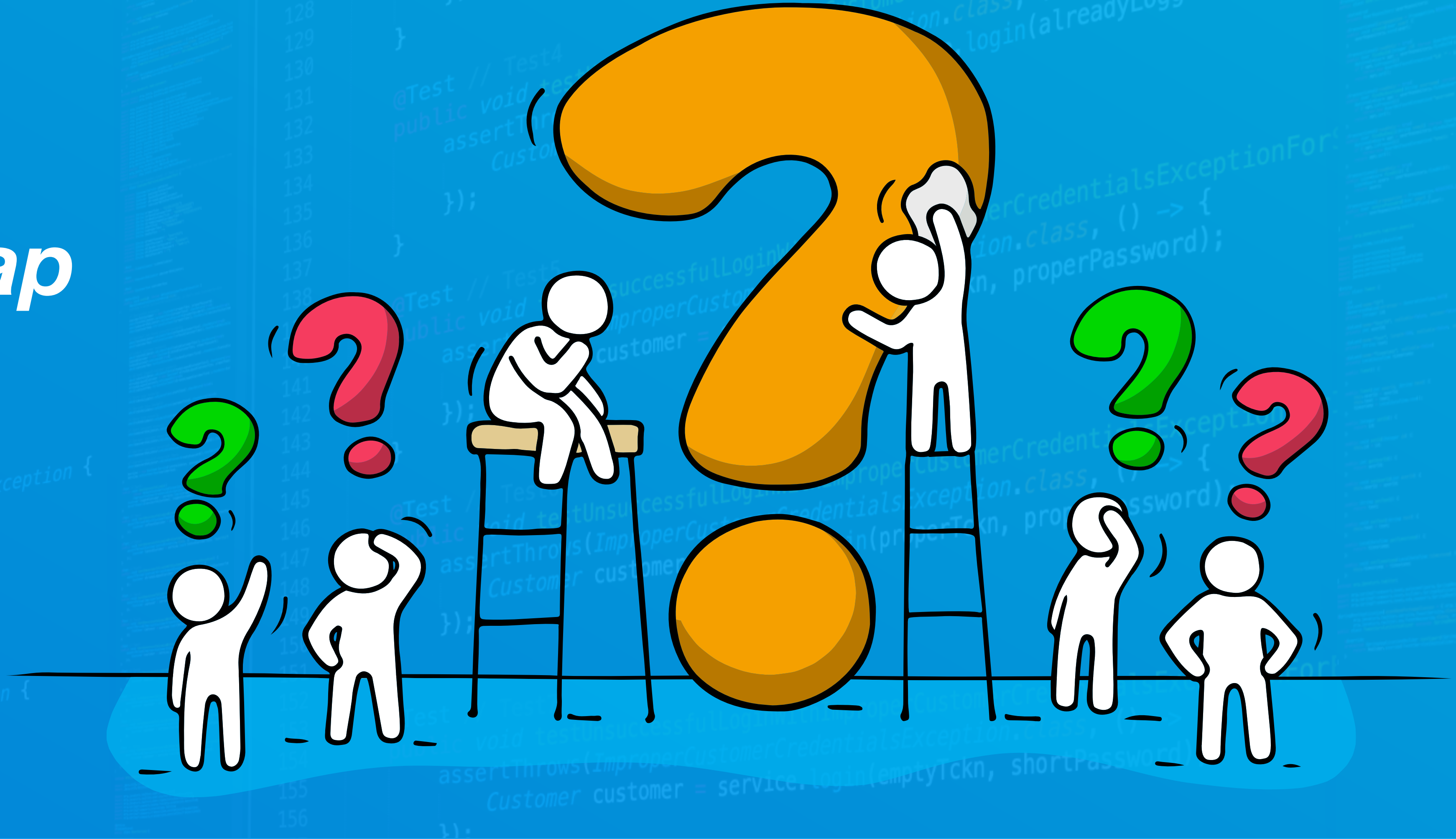
```
class Car {  
    ...  
    String color; // Added later  
    boolean convertible; // Added later  
  
    Car(String newMake, String newModel, String newYear,  
        int newSpeed, int newDistance) {  
        make = newMake;  
        ...  
        distance = newDistance;  
    }  
  
    Car(String newMake, String newModel, String newYear,  
        int newSpeed, int newDistance,  
        String newColor, boolean newConvertible) {  
        make = newMake;  
        ...  
        distance = newDistance;  
        color = newColor;  
        convertible = newConvertible;  
    }  
}
```

Car1, Car2 & Car3



- `problem.car`

Soru ve Cevap Zamanı!





Fake Constructor

Fake Constructor



- Sınıfla aynı isme sahip olup, dönüş değeri de olan bir metot tanımlanırsa derleyici hata vermez ama o metot kurucu değildir
- İsmi kurucu isimdir ama kendisi kurucu olmayan bir metottur, dikkat edin!
- Metot isminin Lower CC olmadığını da fark edin

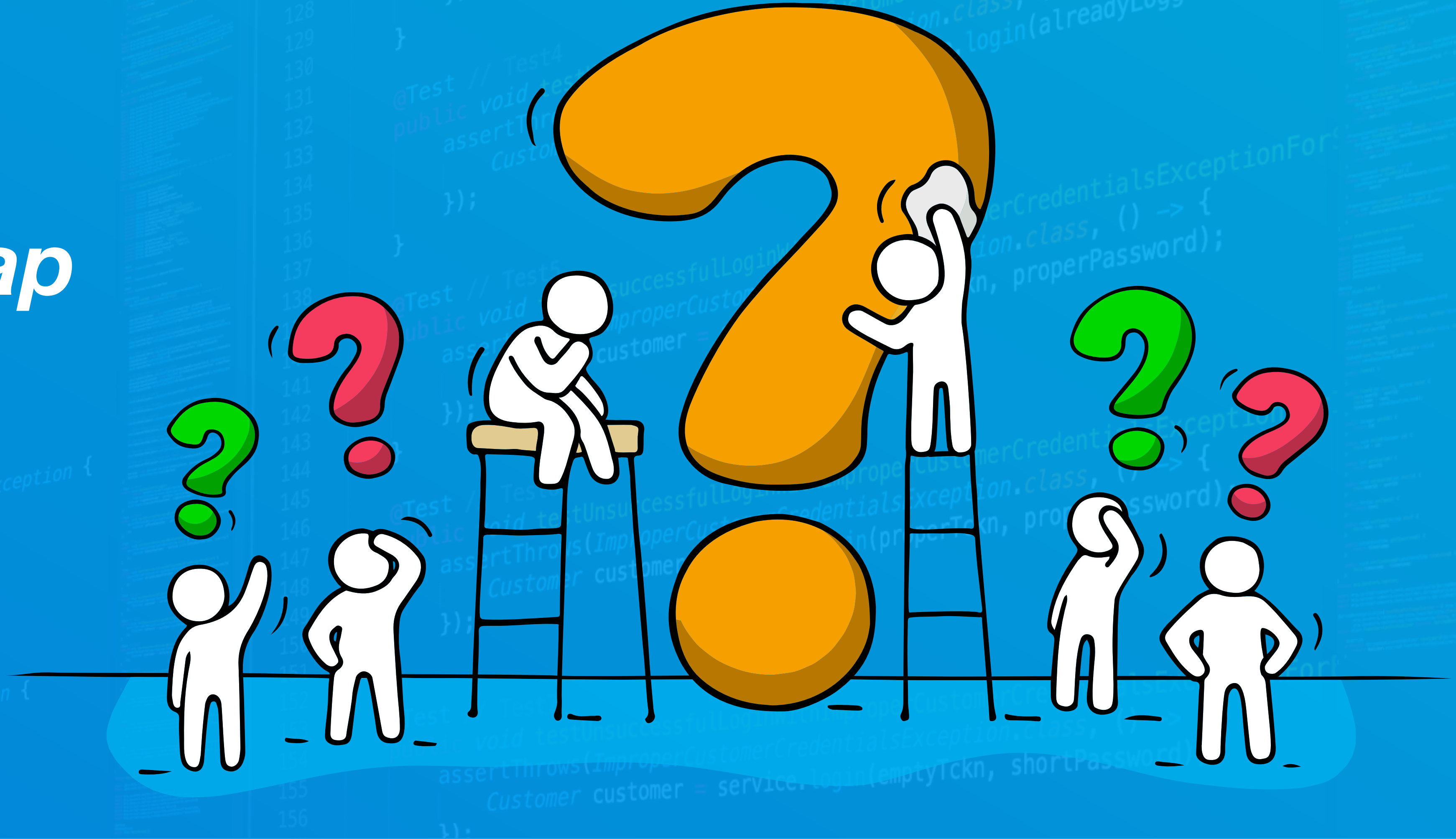
```
class FakeConstructor {  
  
    FakeConstructor() {  
        System.out.println("in FakeConstructor");  
    }  
  
    // Fake constructor!  
    void FakeConstructor() {  
        System.out.println("in FakeConstructor()");  
    }  
  
    void main() {  
        FakeConstructor ref = new FakeConstructor();  
        ref.FakeConstructor();  
    }  
}
```

FakeConstructor



- `problem.FakeConstructor`

Soru ve Cevap Zamanı!





Olusturulmasi Bitmeden Nesneden Hizmet Almak

Nesnenin Durumu ve Hizmet Alma - I



- Durumu son haline gelmeden nesneden hizmet ya da bilgi almayın, alanlarına ilk değer atanmasının bitmesini bekleyin!
- Çünkü bilgi ya da hizmet eksik ya da yanlış olabilir
- Önemli olan nesnenin yaratılmasını tamamlamadan onun alanlarını okumamak!

```
class Tree {  
  
    String type;  
    float height;  
  
    Tree(String newType, float newHeight) {  
        printInfo(); // That's a problem!  
        type = newType;  
        height = newHeight;  
        printInfo(); // That's not a problem!  
    }  
  
    void printInfo() {  
        System.out.println("\nTree Info:");  
        System.out.println("Type: " + type);  
        System.out.println("Height: " + height);  
    }  
    ...  
}
```

Nesnenin Durumu ve Hizmet Alma - II



- Ama örneğin geçilen argümanları kullanarak bilgi verici loglama türünden işleri yapmakta bir sakınca yoktur

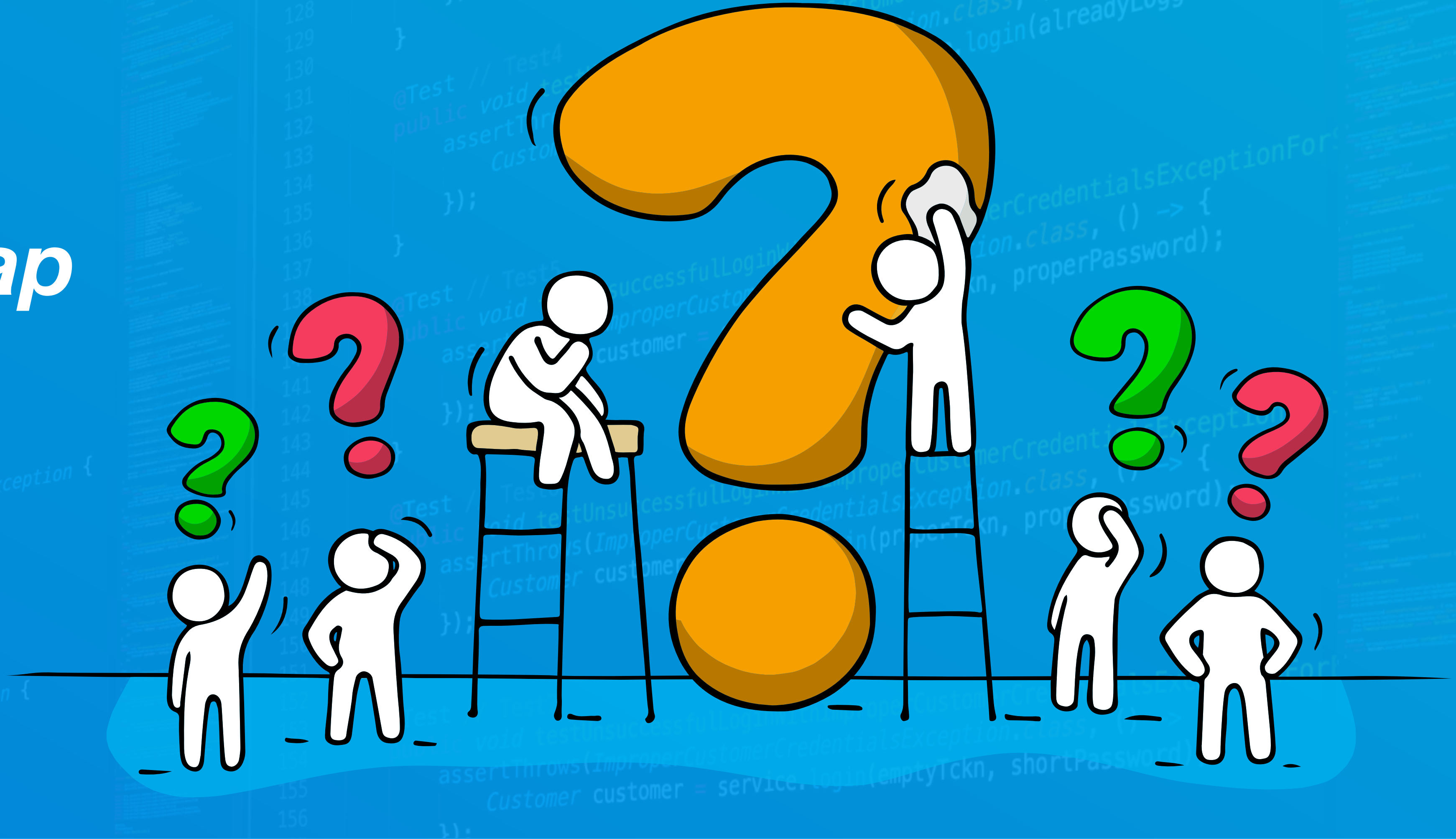
```
class Tree {  
  
    String type;  
    float height;  
  
    Tree(String newType, float newHeight) {  
        // That's ok!  
        log("Creating a tree of type " + newType +  
            " and height " + newHeight);  
  
        printInfo();    // That's a problem!  
        type = newType;  
        height = newHeight;  
        printInfo();    // That's ok!  
    }  
  
    void log(String message){  
        System.out.println("Log: " + message);  
    }  
    ...  
}
```

Tree



- `problem.Tree`

Soru ve Cevap Zamanı!





Referanssız Nesneler

Referanssız Nesneler - I



- Bazen nesne yaratılır ama bir referansa atanmayabilir
- Böyle durumlarda aslolan nesnenin yaratılmış olmasıdır, referansa atanıp daha sonra kullanılması gerekmeyebilir

```
new TreesWithoutReferences("Pine", 5);
```

TreesWithoutReferences



- `TreesWithoutReferences`

Referanssız Nesneler - II



- Bazen kurucu çağrısıyla oluşturulan nesne üzerinde, bir referansa atama yapmadan metot çağrısı yapılır

```
new Employee(...).work();
```

```
Employee e = new Employee(...);  
e.work();
```

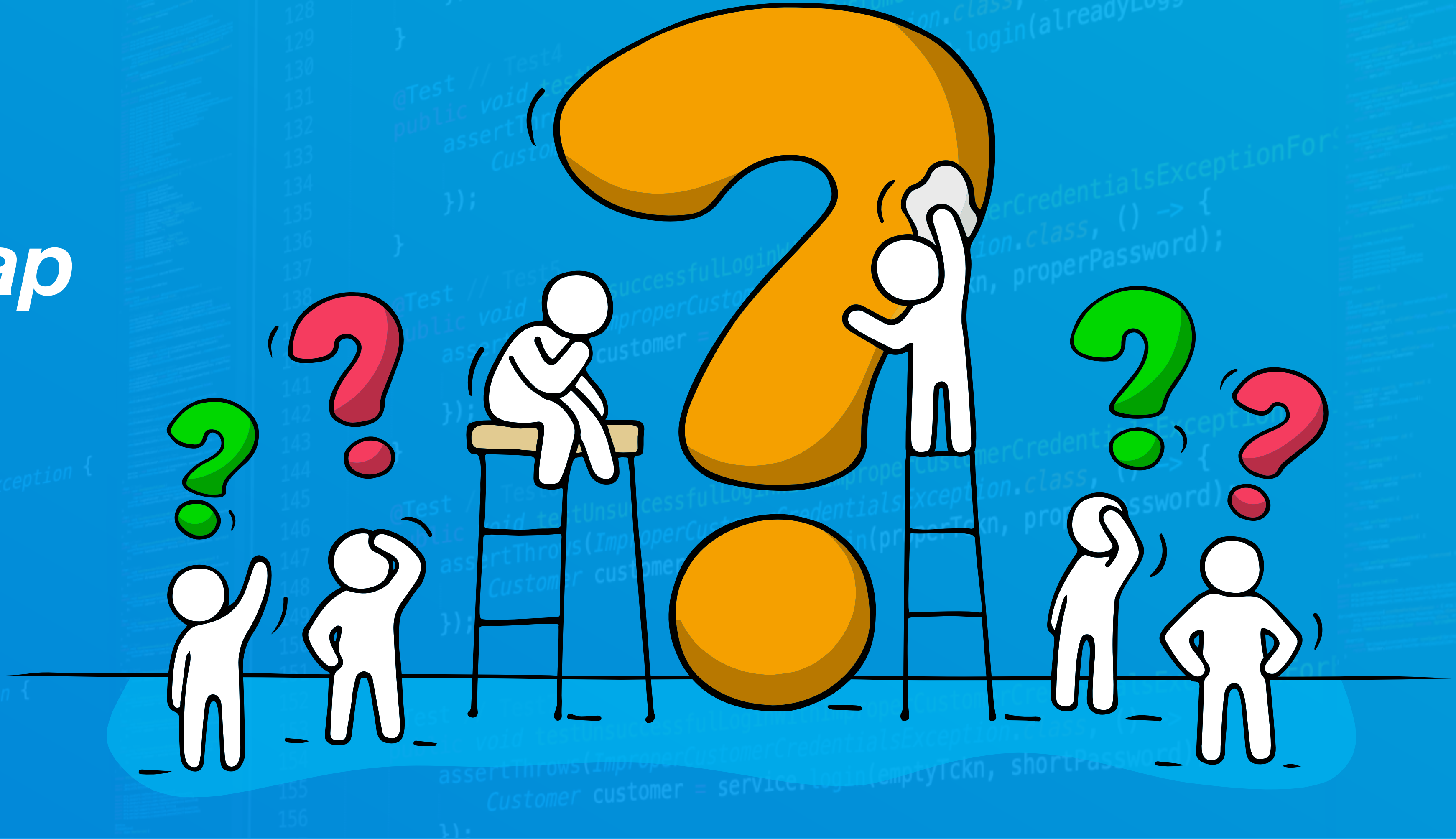
- Amaç nesneyi yaratıp üzerinde tek bir metot çağdırmaksa, kısaca böyle yazılabilir
- Ama aynı nesne üzerinden birden çok metot çağrısı yapılacaksa, nesne bir referansa atanmalıdır

Employee



- Employee

Soru ve Cevap Zamanı!



Ödevler





1. Bir önceki bölümde modellediğiniz köpek ve sahibiyle ilgili oluşturduğunuz sınıflara kurucular ekleyin
 - a. Köpek sınıfına, nesnesini, sahibini geçerek oluşturmanıza izin veren bir kurucu koyun
 - b. Sonra test sınıfını yazıp **main** metodunda nesnelerini oluşturup değerler atayın



2. Bir önceki bölümdeki iki boyutlu bir ortamda bir noktayı temsil eden sınıfa kurucular koyun
 - a. Argüman almayan kurucunun oluşturduğu nesne neyi temsil eder?
 - b. Test sınıfının **main** metodunda, birisi merkezi (origin) temsil eden 3 noktayı kurucu çağrılıyla oluşturup üzerlerinde metot çağrıları yapın



3. Bir önceki bölümde ele aldığınız **Circle/Daire** sınıfına iki tane kurucu koyun:
 - a. Yarıçapı 10 yapan bir varsayılan kurucu
 - b. Geçilen **double** değeri yarıçapa atayan bir akıllı kurucu
 - c. **CircleTest/DaireTest** sınıfında her iki kurucuyu da çağırarak nesneler oluşturup çevre ve alanlarını hesaplatın



4. Bir önceki bölümde modellediğiniz elektrikle çalışan bir lambayı ya da ampülü (bulb) temsil eden sınıfa kurucuları da ekleyin
 - a. Sonra test sınıfını yazıp **main** metotta nesnelerini oluşturup üzerinde metot çağrıları yapın

Bundan Sonra?



Bundan Sonra?



- Bundan sonra `this` anahtar kelimesini ve farklı durumlarda kullanımını göreceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

